

Logbook — Anomalous transport in soaring flights

Matteo Di Sante

Working log

Working notebook. The timeline is regenerated from the git history; the narrative notes are written by hand.

1 Timeline (auto-generated from git)

Date	Event (commit subject)
2026-07-05	thesis ch.4: data-driven pre-processing pipeline + geodesy figures
2026-07-02	Symmetric para/hang dataset; adopt barometric altitude; IGC parser + noise diagnostics
2026-07-01	Rename soaring-ffvl -> soaring-para; add hang-glider data and update all docs
2026-07-01	Fix delta config: season start 2001, data_root /Volumes/SSD_DISANTE/hang_glidern/delta_cfd_igc
2026-07-01	Add soaring-delta CLI for hang-glider CFD data (delta.ffvl.fr)
2026-07-01	Add analysis sub-package and preprocessing diagnostics; revise thesis chapters 1-4
2026-07-01	Expand pre-processing: explicit cleaning and filtering sections
2026-07-01	Clarify stationarity/compatibility check paragraph
2026-07-01	Add preliminary characterization section before main observables
2026-07-01	Add isotropy test protocol to the propagator item
2026-07-01	Rewrite propagator item: formal derivation of marginal scaling
2026-07-01	Reorganize TA-MSD item for clarity and typographic structure
2026-07-01	Clarify ergodicity breaking and add CTRW appendix
2026-06-28	Revise chapter 4 observables and the IGC description
2026-06-28	Warn when thesis and logbook next-steps drift apart
2026-06-28	Align logbook next steps with the thesis planning chapter
2026-06-28	Publish the logbook alongside the thesis
2026-06-28	Add living thesis document and build automation
2026-06-26	Deploy docs via GitHub Pages Actions artifact instead of gh-pages branch
2026-06-26	Fail fast with a clear message when data_root is unset
2026-06-26	Add 'verify' subcommand and remove machine-specific data_root path
2026-06-26	Link published docs in README; drop Development section
2026-06-25	Add GitHub Pages deployment via gh-pages branch
2026-06-25	Translate all documentation, comments, and strings to English
2026-06-25	Initialize thesis repo: FFVL CFD .igc data acquisition

2 Narrative notes

2026-06 — Paraglider data acquisition complete. The full FFVL paraglider CFD archive (`parapente.ffvl.fr`, seasons 1999–2000 to 2025–2026) has been scraped: about 203,000 flight records, of which $\sim 186,000$ carry a GPS track. All 186,052 available tracks were downloaded. *What worked:* impersonating a Chrome TLS fingerprint (`curl_cffi`) reliably passes the FFVL Cloudflare challenge; the resumable, atomic, validated download survived many interruptions without corruption. *What did not / gotchas:* 173 tracks could not be obtained — 172 because the server returns something that is not a valid IGC (broken or placeholder links on FFVL’s side, so a retry does not help), and 1 due to a transient Cloudflare challenge (worth a later retry). On macOS/exFAT, the OS scatters `.*` sidecar files next to every written file; a `clean` step removes them.

2026-06 — Integrity verified. A full on-disk re-scan (`verify`) confirmed that all 186,052 downloaded files are structurally valid IGC: no truncated files, no leftover `.part`, no read errors. The verification logic was promoted from a throwaway script to a first-class `soaring-para verify` subcommand (reusing the download-time validator), with tests.

2026-06 — Robustness of the data path. The external disk remounted under a different name (`HDD_DISANTE` $\rightarrow$ `SSD_DISANTE`), which would have silently broken a hard-coded path. Fix: removed every machine-specific path from the repo (placeholder + `SOARING_PARA_DATA_ROOT` environment variable) and added a start-up guard that aborts with a clear, actionable message when `data_root` is unset/unmounted — and verified that `/Volumes` is not user-writable, so a wrong path can never cause a silent download to the wrong place.

2026-07 — Hang-glider data acquired; naming corrected. The hang-glider CFD (`delta.ffvl.fr`, seasons 2001–2002 to 2025–2026) was added as a second source. The acquisition code (`soaring.acquisition.ffvl`) is shared; a second CLI entry point `soaring-delta` (config `delta_download.yaml`, env var `SOARING_DELTA_DATA_ROOT`) drives it. Data: $\sim 9,300$ flights, 6,716 tracks downloaded, 30 failures (broken server-side links, same pattern as paragliders). Stored on the SSD at `/Volumes/SSD_DISANTE/hang_gliders/delta_cfd_igc/`. At the same time the paraglider CLI was renamed from the generic `soaring-ffvl` to `soaring-para` (and `SOARING_FFVL_DATA_ROOT` $\rightarrow$ `SOARING_PARA_DATA_ROOT`), and the config was renamed from `ffvl_download.yaml` to `para_download.yaml`, to avoid ambiguity now that both sources are present.

2026-07 — Hang-glider catalog built; datasets reorganized symmetrically. Ran the full pipeline on the hang-glider data (`soaring-delta clean, verify, build-catalog, status`), which so far had only been downloaded: 9,259 flights, 6,746 with a track, 6,716 downloaded and all verified (0 integrity failures), 30 unrecoverable server-side links. The local `data/` folder was reorganized to mirror the SSD — one directory per discipline (`paragliders/`, `hang_gliders/`), each with a versioned `seasons_index.csv` and a git-ignored `catalog.csv`. `generate_stats.py` now reads both indices and emits per-discipline macros (`\StatPara*`, `\StatHang*`) and two per-season tables, and the first three thesis chapters were made symmetric across the two disciplines.

2026-07 — Altitude channel decision: barometric. Decided to take the vertical dynamics from the *barometric* altitude rather than the GNSS altitude, diverging from the reference pipeline (J  r  mie’s code derives z and v_z from `AltGNSS`, reading but not using `AltPre`). Rationale: the pressure altitude has sub-metre relative precision and low high-frequency noise, its errors being slow (offset, drift) and killed by differencing, whereas the GNSS vertical carries

a high-frequency noise floor that derivatives amplify. Confirmed empirically with a new noise diagnostic – an IGC B-record parser (`soaring.analysis.igc`) and an altitude-noise analysis (`soaring.analysis.altitude_noise`) whose Welch power spectra show the GNSS floor two to three orders of magnitude above the barometric one near Nyquist (with a thesis appendix on the method). *Consequences*: the A/V validity flag is not a separate rule but is subsumed by the missing-altitude check on the adopted channel (a V fix keeps position + baro on a baro flight, but is dropped on a GNSS-fallback flight); one altitude channel per trajectory, no baro/GNSS splicing; flights with no pressure sensor fall back to unfiltered GNSS, tagged by a per-flight `alt_source` column (fractions — a substantial minority, larger for paragliders — to confirm on a larger sample once the SSD is remounted); the trimming threshold is on the horizontal speed only ($v_{xy} > 5$ m/s, sustained); and the cleaning/trimming run on the raw geographic coordinates (great-circle speeds), with the ENU conversion done *last*, origin at take-off. Also redesigned the noise figure (dropped the redundant HF-index panel; panel (a) now shows the GNSS–baro difference).

2026-07 — Baro-absence fraction: sized sampling, not a full sweep. A full-population parse of the paraglider archive (186,052 files) for the baro-channel-absence figure took an unacceptable ~ 45 min serially; a first fix (parallelising across 8 processes) cut this to ~ 15 min, still too slow to be run routinely. Replaced it with a properly justified *simple random sample*: the standard proportion sample-size formula ($n = z^2 p(1-p)/e^2$, conservative $p = 0.5$) gives $n = 2401$ for a target 95%-confidence margin of error of ± 2 percentage points – `required_sample_size` in `soaring.analysis.altitude_noise`, with `proportion_ci` reporting the resulting point estimate and interval. The population is treated as effectively infinite (no finite-population correction): at $n = 2401$ out of 186,052 paraglider flights ($\sim 1.3\%$ sampled), the correction would move n by only about 30, not worth the extra parameter. The hang-glider archive (6716 files) stays a full census: cheap enough (under two minutes) that there is nothing to gain from sampling it. Total runtime: ~ 50 s. Final figures: paragliders $(28.4 \pm 1.8)\%$ (95% CI, sampled), hang gliders 17.5% (exact, census) – consistent with the earlier, less rigorously sized estimates. *Also caught mid-flight*: the IGC parser accepted syntactically well-formed but semantically invalid B records (e.g. minute ≥ 60 , hour ≥ 24 , out-of-range latitude/longitude) without rejecting them; added explicit format-validity checks (9 new tests) and corrected the thesis wording that had conflated this *implemented* format-validity layer with the fix-level *dynamics-plausibility* layer of Table 4.1, which is designed (thresholds exist as a dataclass) but not yet implemented as code – an important distinction the thesis now states explicitly rather than glossing over.

2026-07 — Pre-processing made data-driven and config-driven. Reworked Chapter 4 so the flight-level filtering is computed from the *full parsed dataset* (paragliders and hang gliders), not the declared catalog metadata: a track scan (`soaring.analysis.preprocessing.scan_tracks`) parses every downloaded track, and Figure 4.2 shows the real duration and flown-path-length distributions with per-variable *marginal* retention curves. The cuts are duration ≥ 40 min and path length ≥ 30 km (a minimal cut, $\sim 95\%$ retained); the earlier min-fix-count cut was dropped as redundant with the duration cut, and the *extent* proxy was replaced by the total flown *path length* (median 86 km para, 158 km hang). *Key decision*: every pre-processing threshold now lives in `configs/preprocessing.yaml` (never hard-coded), loaded via `load_preproc_config` into typed dataclasses. Also added: the great-circle (haversine) formulas giving inter-fix speed and distance *before* the ENU conversion; a Savitzky–Golay smoothing/differentiation section with a noise-matched procedure for its two hyperparameters; and the switch from a common resampling cadence to native per-flight Δt (Figure 4.5). An implementation blueprint (`docs/guide/preprocessing-pipeline.md`) records the exact schema and steps, to retire into code once the pipeline is built.

2026-07 — Gap diagnostics; scan cache; baro-presence consolidated. Added Figure 4.5, the sampling-regularity diagnostics (largest gap relative to native Δt , missing fraction of a uniform grid, each with its marginal retention curve), fulfilling a promise Sec. 4.1.6 already made (“its retained fraction reported with the others”) but had not yet delivered. Since a fresh full parse for this would have repeated the ~ 15 – 20 min paraglider census just done for Figure 4.2, the per-flight scan is now **cached** to `data/<discipline>/track_scan.parquet` (gitignored, like `catalog.csv`; `load_or_scan_tracks` reads it if present, else scans and writes it) – later moved to the SSD, see below. While at it, `track_stats` was extended with a few near-free byproducts of the same scan – barometric-presence fraction, max horizontal/vertical speed, barometric altitude range – available for future validation of Table 4.1’s fix-level bounds against real data.

Consolidation: the barometric-presence fraction (Sec. 4.1.1) now reads from this same cache (`altitude_noise.baro_presence_from_scan`) instead of running its own separate scan, turning the paraglider number from a sized sample into an exact census at no extra cost: 30.1 % (exact, $n = 186,025$) – consistent with the earlier sampled estimate, $(28.4 \pm 1.8) \%$, which correctly bracketed it. *Also caught:* a pre-existing fragility in `altitude_noise.collect` where a single unreadable file (a transient external-disk hiccup surfaced by two heavy scans back to back) aborted the whole PSD sample; it now skips and logs the file instead.

2026-07 — SSD reorganised; no data kept locally; figure fixes. *Policy going forward:* raw data, catalogs and every analysis byproduct live only on the external SSD, never in the local repo. Reorganised both discipline’s `data_root` by maturity – `raw/{igc,raw_xml}` (untouched acquisition output), `catalog/{catalog.csv,seasons_index.csv}` (tables derived from it), `derived/` (further byproducts, today just `track_scan.parquet`; future cleaned/filtered data and analysis results will get their own `processed/` and `analysis/` siblings) – and updated `Config` accordingly (new `derived_dir` property). Verified file counts before/after the move on both SSDs (paraglider: 186,052 tracks, 27 XMLs; hang glider: 6,716 tracks, 25 XMLs – all matched) and that both acquisition CLIs (`soaring-para/soaring-delta status`) still work against the new layout, since every acquisition module already went through `Config`’s path properties with no hard-coded paths elsewhere. The previously-local `track_scan.parquet` copies moved to `<data_root>/derived/` (read back and row/column-checked after the move); the orphaned local `catalog.csv` copies (superseded by the track-based scan months ago, confirmed unreferenced by any code) were removed.

Figure fixes, from a closer look at the newly-cached data. Figure 4.5 (gaps): the adopted `max_gap_factor` was raised from 5 to 10 – at 5 it retained only 77.8 %/65.0 % of para/hang flights (nowhere near the near-full retention of the duration/path cuts), at 10 it reaches 86.9 %/85.1 %. Considered 20 (90.8 %/90.6 %, little further gain) and rejected it: the cut is *relative* to each flight’s own native Δt , so for the slower hang-glider loggers (up to 10 s native) a factor of 20 would tolerate interpolating across a ~ 200 s gap – long enough to span a thermal climb, not a sane linear-interpolation span. Also tightened the distribution panels’ axis range (most mass sits under the 90th percentile; the old 0–200/0–0.6 range was mostly empty). Figure 4.1 (altitude noise): panel (a)’s difference trace now uses a colour distinct from both channels (was reusing GNSS’s red); its window grew from 10 to 30 minutes; and a Nyquist-frequency line was added to panel (b) (previously discussed in the caption but never drawn). Along the way, found and fixed a real bug in the “representative flight” selection: it picked the first flight matching the PSD criteria, later changed to the *longest* by elapsed time – until a 30-hour, 1,100-fix outlier (one corrupted timestamp gap) exposed that elapsed time is not robust to a single bad gap. Fixed to select by *fix count* instead, which does not have this failure mode.

2026-07 — Fix-level cleaning made data-driven; parser roll-over bug; figure and data-folder cleanups. *Fix-level cleaning* (Table 4.1) is now audited on the data, like the flight-level cuts. Added a figure to Sec. 4.1.2 (fix-level cleaning): the per-fix distributions of horizontal speed,

barometric vertical speed and barometric altitude (`make_fixlevel_diagnostics_figure`, from `fix_level_distributions` over a seeded per-fix sample – millions of fixes). The point that is easy to get wrong: a fix-level bound removes single fixes from an otherwise good flight, so what justifies it is the fraction of *fixes* it removes, not the number of flights it touches (a bound may touch many flights harmlessly). Each bound sits in the sparse, implausible tail. The per-flight *maxima* already in the scan (`max_vxy_mps` and friends) answer the wrong question – flights touched – so they do not place the cuts.

Removed the absolute 60 s inter-fix time-gap from the fix-level table. It duplicated the sampling-regularity cut of Sec. 4.1.6, which is more principled: relative to each flight’s own native Δt , so it scales with the logger’s rate. Gaps are now handled in exactly one place. Neither was ever an enforced filter (both are declared thresholds for the still-to-build cleaning routine), so this is a spec simplification, not a behaviour change.

Found and fixed the root cause of the “30-hour, 1,100-fix” pathology worked around in the previous entry. The real bug was in the IGC parser’s midnight roll-over unwrap, which treated *any* backward step in the time-of-day as a new day and added 86,400 s: a few-second GPS glitch became a spurious multi-hour gap and a wildly wrong duration. Now only a drop of (nearly) a whole day counts as a roll-over, and residual backward jitter is clamped so the series stays monotonic. Added regression tests for both the true roll-over and the glitch.

Figure and config cleanups. The altitude-noise figure (Sec. 4.1.1) is now four panels: (a) the two raw channels overlaid on one window – they track the same climb, GNSS visibly noisier – beside (b) their difference, then (c) the PSD and (d) the fallback rate; this also fixed the old a/b/d panel labelling. The gap-diagnostics figure (Sec. 4.1.6): the distribution panels no longer clip the tail into an artificial spike at the edge, and their x-range is fitted to the data’s own percentiles instead of a fixed constant. On principle – the code runs on datasets of different sources and rates – result-specific numbers (retention percentages, one-off anecdotes) were pruned from the YAML and code comments; such numbers belong in the thesis, not in reusable config.

Local data folder clarified. The two versioned `seasons_index.csv` are the offline reproducibility anchor for the thesis’s dataset counts – they let the document build with no SSD mounted – not stray local data. Their sync from the canonical SSD copy is now explicit and automatic (`refresh_seasons_index.py`, run best-effort by the pre-commit hook), so the snapshot cannot silently drift; running it immediately caught a stale paraglider snapshot (only the line endings differed – the counts were identical). Rewrote `data/README.md` and fixed several stale references to a flat `data/seasons_index.csv` path that never existed.

2026-07 — Coordinate/geodesy figures. Redrew the coordinate-transformation figures: Figure 4.3 (WGS84 ellipsoid; geodetic vs geocentric latitude) and Figure 4.4 (the ECEF→ENU rotation, in our notation), and added the ESA Navipedia citation for the ECEF conversion.

2026-06 — Documentation. Project docs are built with MkDocs and published on GitHub Pages. Initially deployed with `mkdocs gh-deploy`, which accumulated one built-site commit per deploy on a `gh-pages` branch; migrated to the official GitHub Pages *artifact* workflow (build → upload artifact → deploy), removing the branch and keeping the git history clean.

Operational note. In this repo `uv sync` occasionally fails to (re)install the editable package; `uv sync --reinstall-package soaring` fixes it.

3 Next steps

These mirror the planning chapter of the state document (`thesis/sections/04-next-steps`) and are kept in sync with it. The near-term plan is to analyse the data *in parallel* with studying the segmentation, since many observables can be built before any phase splitting.

- **Data sources.** FFVL paraglider and hang-glider CFD data are collected, cataloged and verified. Extend to further sources – notably WeGlide for sailplanes (a higher-quota request is pending) and possibly XContest – merging into one catalog and stratifying by source. Refresh the per-discipline `seasons_index.csv` via `build-catalog` so the document statistics stay current.
- **Pre-processing.** The parser is in place (`soaring.analysis.igc`) and the pipeline is designed and thresholded (`configs/preprocessing.yaml`): fix-level cleaning (great-circle speeds), v_{xy} -based trimming, data-driven flight-level filtering (duration + path length), ENU conversion (barometric vertical), native- Δt uniformity, and Savitzky–Golay smoothing/differentiation. Next: implement it end-to-end (parser $\rightarrow \dots \rightarrow$ Parquet `fixes` table) per the blueprint, and measure the ENU power spectra to fix the Savitzky–Golay window.
- **Whole-trajectory observables (no segmentation).** Immediate next step: characterise the data with observables that need no phases — MSD and its scaling exponent, the displacement distribution and its tails, the velocity autocorrelation and turning-angle distribution, the speed/vertical-velocity distributions, and geometric measures (tortuosity, fractal dimension, radius of gyration). Goal: a first physical/statistical picture and some preliminary findings.
- **Segmentation.** Study Jérémie’s internship report and code, then re-examine the transition/search/climb HMM (binary features: sign of vertical speed, curvature-angle persistence, straightness) — decide whether to keep that approach or add complexity for a better segmentation on our larger, more heterogeneous data.
- **Phase-resolved observables.** After segmentation, extract step lengths, waiting times, search durations, per-phase kinematics and the angular diffusion between successive glides.
- **Reproduce the reference.** Check that the enlarged dataset reproduces Vilpellet et al.: per-phase conditional MSD (their Fig. 3) and per-phase descriptive statistics (their Fig. 4), plus the global $H \approx 0.88$ — validation + universality test.
- **Transport analysis.** Estimate the MSD scaling exponent, fit the tails of the waiting-time and step-length distributions, and perform CTRW-vs-Lévy-walk model selection.
- **Minimal model.** A minimal stochastic model for $H \approx 0.88$: resolve transition segments, merge search+climb into one effective state, with angular diffusion between successive transition directions (preliminary idea, to be refined/changed).